Phonetic Contrastive Model — Technical Document

Project: Roman Urdu ASR Post-Processing **Module:** `qwen3-asr-local/phonetic_contrastive_model/`
Date: July 2026 **Version:** v1

Table of Contents

1. Executive Summary

The Phonetic Contrastive Model is a character-level Siamese bi-encoder neural network designed to normalise the spelling of Roman Urdu words in ASR (Automatic Speech Recognition) transcriptions. It maps phonetically similar but orthographically diverse spellings to their canonical forms — for example, mapping "chugataai", "chugatai", and "chughtaai" to the single canonical "Chughtai".

Unlike the hand-crafted rule-based resolver it replaces, the model **generalises to unseen spellings** — it achieves ~97% held-out recall versus the resolver's ~30%. It integrates as an optional fallback layer (Layer 4b) in the `hindi_to_roman_urdu.py` transliteration pipeline, activated by setting the environment variable `PHONETIC=1`.

The model is lightweight (~3.2 MB checkpoint), runs on CPU with sub-millisecond per-word latency, and includes a built-in abstain mechanism that prevents it from corrupting words it is not confident about.

2. Problem Statement

2.1 The Roman Urdu Spelling Challenge

Roman Urdu has no standardised orthography. The same word can be spelled dozens of different ways by different speakers:

Canonical	Observed Variants
Siddiqui	siddiqee, siddiqi, siddique, sidiqui, siddiki
Chughtai	chugataai, chugatai, chughtaai, chughati
operation	apareshan, opreshan, aperation, opration
hospital	haspatal, hispatal, hospitaal, haspitaal

When the ASR system (Qwen3) transcribes Urdu audio, it produces Hindi (Devanagari) text. This is then transliterated to Roman Urdu using rule-based phoneme mapping. The resulting Roman Urdu is phonetically correct but may not match the expected "canonical" spelling that downstream systems or human readers expect.

2.2 Why the Rule-Based Approach Fails

The previous solution (the "resolver" in `resolver.py`) used edit-distance matching with phonetic substitution rules (z↔j, aa↔a, ee↔i, etc.). This approach had two fundamental problems:

1. **Poor coverage:** Only ~30% held-out recall. Every new spelling variant required manually adding substitution rules — an $O(N^2)$ maintenance burden.
2. **No generalisation:** The rule engine could only match patterns someone had explicitly programmed. Novel garbles (never seen before) were missed.

2.3 Requirements

- **High recall:** Correctly normalise ≥90% of unseen spelling variants
- **High precision (abstain safety):** Never corrupt words that are already correct
- **Low latency:** Sub-millisecond per word on CPU
- **Maintainability:** Adding new canonical terms should not require retraining
- **Self-contained:** Single checkpoint file with all state — no external dependencies

---

3. Solution Overview

The Phonetic Contrastive Model is a **metric learning** approach: it learns an embedding space where phonetically equivalent words cluster together, regardless of their surface spelling. At inference, an unknown word is embedded and compared against a pre-computed index of all canonical embeddings. If the nearest canonical is close enough (cosine similarity ≥ 0.90), it is returned as the correction; otherwise the word is left unchanged (abstain).

3.1 Key Design Principles

Principle	Implementation
Character-level processing	Operates on individual characters, not word-level tokens
Siamese architecture	Same encoder for variants and canonicals
Contrastive learning	InfoNCE loss pushes variants toward their canonical
Abstain-over-corrupt	0.90 cosine threshold — uncertain → do nothing
Pre-computed index	Canonical embeddings computed once, stored in checkpoint
Runtime extensibility	New canonicals onboarded without retraining

---

4. Model Architecture

4.1 CharEncoder — The Core Network

The model is a `CharEncoder` class implementing a character-level bidirectional GRU with dual pooling and a projection head. The architecture processes arbitrary-length character sequences into fixed-size, L2-normalised embeddings.

4.1.1 Character Vocabulary

Characters are represented using a simple vocabulary:

```
Vocabulary = [<pad>, <unk>, a, b, c, ..., z]  (~30 tokens)
```

- `<pad>` (index 0): Padding token for batch alignment
- `<unk>` (index 1): Unknown character fallback
- All input is lowercased before encoding
- Maximum sequence length: 40 characters (truncated)

The vocabulary is built from all training strings (variants + canonicals) and stored in the checkpoint as `itos` (index-to-string mapping).

4.1.2 Embedding Layer

```
nn.Embedding(vocab_size, emb_dim=96, padding_idx=0)
```

Each character is mapped to a dense 96-dimensional vector. The padding index ensures `<pad>` tokens produce zero vectors, preventing them from contributing to downstream computations.

4.1.3 Bidirectional GRU

```
nn.GRU(
    input_size=96,       # emb_dim
    hidden_size=128,     # per direction
    num_layers=2,        # stacked
    batch_first=True,
    bidirectional=True,  # output: 128 × 2 = 256 per timestep
    dropout=0.2          # between layers
)
```

The GRU reads the character sequence in both forward and backward directions. Each timestep produces a 256-dimensional output (128 per direction). The bidirectional design is critical: phonetic similarity often depends on characters at both ends of a word (e.g., "siddiq**ui**" vs "siddiq**ee**").

Why GRU over LSTM? GRU has fewer parameters (no separate cell state), trains faster on short sequences, and achieves equivalent performance for character-level tasks where long-range dependencies are limited.

Why not Transformer? The inputs are short (average ~8 characters). Transformers' $O(T^2)$ attention is unnecessary overhead for sequences this short, and the GRU's inductive bias for sequential character processing is a natural fit.

4.1.4 Masked Dual Pooling

After the GRU, the model aggregates timestep outputs into a single fixed-size vector using **masked mean + max pooling**:

```
# Mask out padding positions
mask = (ids != pad_idx).unsqueeze(-1)        # (B, T, 1)
out = gru_output * mask                      # zero the pad steps

# Mean pooling: sum / count (excluding padding)
summ = out.sum(dim=1)                        # (B, 256)
cnt = mask.sum(dim=1).clamp(min=1)           # (B, 1)
mean = summ / cnt                            # (B, 256)

# Max pooling: max over non-padded positions
very_neg = torch.finfo(out.dtype).min
mx = out.masked_fill(~mask, very_neg).max(dim=1).values  # (B, 256)

# Concatenate
pooled = torch.cat([mean, mx], dim=-1)       # (B, 512)
```

Why dual pooling? - **Mean pooling** captures the average character pattern — the "centre of mass" of the character embeddings. This is robust to local character-level noise. - **Max pooling** captures the most salient character features — if any position has a distinctive character pattern, it will be preserved. This helps the model detect discriminative character n-grams.

The concatenation of both produces a 512-dimensional representation that is both robust (mean) and discriminative (max).

4.1.5 Projection Head

```
nn.Sequential(
    nn.Linear(512, 128),    # pooled_dim → out_dim
    nn.LayerNorm(128)       # stabilise embedding scale
)
```

The linear projection reduces dimensionality from 512 to 128, and `LayerNorm` ensures embeddings have consistent scale regardless of input. This is crucial for the cosine similarity comparison at inference — without normalisation, some inputs might produce larger-magnitude embeddings than others, biasing the similarity computation.

4.1.6 L2 Normalisation

```
z = F.normalize(z, dim=-1)    # → unit-norm vectors on the 128-d hypersphere
```

The final output is L2-normalised, placing all embeddings on the surface of a 128-dimensional unit hypersphere. This has two benefits:

1. **Cosine similarity = dot product:** `cos(a, b) = a · b` when both are unit-norm. This makes nearest-neighbor search a simple matrix multiplication.
2. **Scale invariance:** The model cannot "cheat" by making confident predictions larger. All similarity scores are in [-1, 1].

4.2 Complete Forward Pass Summary

```
Input: "chugataai" (string)
  ↓ Vocab.encode() — lowercase, char→id, pad to batch max
[3, 8, 21, 7, 1, 20, 1, 1, 9]  (character indices)
  ↓ nn.Embedding(96)
(1, 9, 96)  — 9 characters, 96-dim each
  ↓ Bidirectional GRU (2 layers, 128 hidden)
(1, 9, 256) — 256 = 128 forward + 128 backward
  ↓ Masked mean pooling → (1, 256)
  ↓ Masked max pooling  → (1, 256)
  ↓ Concatenate          → (1, 512)
  ↓ Linear(512→128) + LayerNorm(128)
(1, 128)
  ↓ L2-normalise
(1, 128)  — unit-norm embedding vector
```

---

5. Training Methodology

5.1 Training Data

The training data comes from `data/lexicons_v2.json`, a manually curated lexicon of Roman Urdu words:

```
{
  "lexicons": {
    "lexicon": {
      "Siddiqui": ["siddiqee", "siddiqi", "siddique", ...],
      "Chughtai": ["chugataai", "chugatai", "chughtaai", ...],
      "operation": ["apareshan", "opreshan", "aperation", ...],
      ...
    }
  }
}
```

Each entry maps a **canonical** (correct spelling) to its known **variants** (garbled spellings). Only single-word entries are used (multi-word phrases are excluded from training).

5.2 Data Splitting

The `make_splits()` function creates reproducible splits with seed=13:

Split	Fraction	Purpose
Train	~72%	Training pairs (variants the model sees)
Val	~8%	Early stopping — carved from train set
Heldout	~20%	Generalisation test — NEVER seen during training

Critical constraint: Every canonical retains at least 1 variant in the training set. This ensures the canonical is "known" during training while the held-out variants are truly unseen.

5.3 Contrastive Learning with InfoNCE

The model is trained using **InfoNCE (Information Noise-Contrastive Estimation)** loss, a contrastive objective from the metric learning literature.

5.3.1 Batch Construction

For each mini-batch of variant-canonical pairs:

1. **Variants (anchors):** B variant strings are encoded: `v_emb = encoder(variants)` → (B, 128)
2. **Candidates (positives + negatives):** The batch's unique true canonicals PLUS a random sample of 256 other canonicals are encoded with the same encoder: `c_emb = encoder(candidates)` → (U+256, 128)

Key design choice: All candidates (positives and negatives) are encoded **fresh** with the current model weights on every forward pass. This avoids the "stale bank" problem where negative embeddings are computed with older model weights and become inconsistent with the positive embeddings.

5.3.2 Loss Computation

```
logits = v_emb @ c_emb.T / temperature    # (B, U+256)
loss = CrossEntropy(logits, target)
```

Where: - `temperature = 0.07` (sharpens the softmax distribution) - `target[i]` = index of variant i's true canonical in `c_emb`

The loss encourages: - **High similarity** between a variant and its true canonical (the positive) - **Low similarity** between a variant and all other canonicals (the negatives)

5.3.3 Why Unique Canonicals Matter

If two variants in a batch share the same canonical (e.g., "siddiqee" and "siddiqi" both map to "Siddiqui"), treating each canonical copy as a negative for the other variant would create **false negatives**. Using unique canonicals in the candidate set eliminates this problem.

5.4 Optimisation

Component	Setting
Optimiser	AdamW (lr=1e-3, weight_decay=1e-4)
Learning rate schedule	CosineAnnealingLR (T_max = max_epochs)
Gradient clipping	Max norm = 5.0
Batch size	256
Max epochs	100
Early stopping	Patience = 5 epochs without val_recall improvement

5.5 Validation and Early Stopping

After each epoch, the model computes **val_recall**: top-1 nearest-canonical accuracy on the validation set. The procedure:

1. Embed all canonicals → (N, 128) matrix
2. Embed all validation variants → (V, 128) matrix
3. Compute similarity: `sims = val_emb @ canon_emb.T` → (V, N)
4. For each variant, check if `argmax(sims)` matches the true canonical
5. `val_recall = correct / total`

The model state with the highest val_recall is saved. Training stops early if val_recall does not improve for 5 consecutive epochs.

5.6 Post-Training Index Construction

After training completes (or early-stops), the best model state is restored and used to embed ALL canonicals:

```
canon_emb = embed_all(model, canonicals, vocab, device, batch=512)
```

This produces an (N, 128) matrix of L2-normalised canonical embeddings. This matrix is saved in the checkpoint alongside the model weights, so inference never needs to re-encode the canonicals.

---

6. Inference Pipeline

6.1 Loading

```
corrector = PhoneticContrastiveCorrector.load()
```

The `load()` class method: 1. Loads the checkpoint file (`phonetic_contrastive_v1.pt`) 2. Reconstructs the `Vocab` from the saved `itos` 3. Reconstructs `CharEncoder` from the saved config 4. Loads model weights from `state_dict` 5. Sets model to `eval()` mode (dropout off) 6. L2-normalises the saved canonical embeddings

6.2 Word Resolution

```
result = corrector.resolve_word("chugataai")  # → "Chughtai"
```

The resolution algorithm:

```
1. IF word is non-alphabetic           → return unchanged
2. IF word.lower() in known_canonicals → return unchanged (already correct)
3. IF len(word) < 3                    → return unchanged (too short to trust)
4. Encode word: q = model(word)        → (1, 128) unit-norm vector
5. Compute similarities: sims = q @ index.T  → (N,) cosine scores
6. Find best: best_idx = argmax(sims), best_score = sims[best_idx]
7. IF best_score < threshold (0.90)    → return unchanged (ABSTAIN)
8. ELSE                                → return canonicals[best_idx]
```

6.3 Text Resolution

```
result = corrector.resolve_text("yeh chugataai lab ka kaam hai")
```

Uses regex `[A-Za-z]+` to find all alphabetic tokens, applies `resolve_word()` to each, and returns the text with corrections applied.

6.4 Case Handling

When the model returns a canonical, it respects the original capitalisation: - If the input starts with a capital and the canonical is all-lowercase, the output capitalises the first letter. - If the canonical has inherent capitalisation (e.g., "CNIC", "Chughtai"), it is returned as-is.

---

7. Safety Mechanisms

7.1 The Abstain Threshold

The most critical safety feature. When the model's best cosine similarity score falls below the threshold (default 0.90), it returns the input word unchanged rather than guessing.

Why 0.90? This value was determined by sweep evaluation: - At 0.90, the model achieves high recall on genuine garbles while leaving nearly all real words unchanged. - Lower thresholds increase recall but start corrupting legitimate words. - Higher thresholds reduce recall unnecessarily.

7.2 Known-Canonical Short Circuit

If a word is already in the canonical set, the model skips encoding and similarity computation entirely. This prevents a canonical from being "corrected" to a different canonical that happens to have a higher similarity score.

7.3 Minimum Length Guard

Words shorter than 3 characters are never processed. Short words (e.g., "is", "ka", "ke") have too little character signal for reliable matching and are more likely to be false positives.

7.4 Statistics Tracking

The corrector maintains per-session statistics:

```
stats = {
    "exact": 0,          # exact lexicon matches (upstream)
    "matched": 0,        # successfully resolved by model
    "abstain_short": 0,  # skipped: too short
    "abstain_low": 0,    # skipped: below threshold
    "already": 0         # skipped: already a canonical
}
```

---

8. Extensibility — Onboarding New Canonicals

8.1 Runtime Addition (No Retraining)

```
corrector.add_canonical("XYZlab")
```

1. Encodes the new term with the existing model
2. Appends the embedding to the index
3. Adds the term to the known set

This works because the model has learned a general character-level embedding space. A new canonical placed in this space will "attract" its phonetic variants — even ones never seen during training.

8.2 Bulk Extension via `extend_canonicals.py`

For production use, the `extend_canonicals.py` tool:

1. Reads new entity terms from a text file
2. Adds each to the model's index
3. Validates each against the 80-call benchmark
4. Drops terms that cause wrong captures (ambiguous terms)
5. Saves the extended index to the checkpoint

6. Generates an updated lexicon (e.g., `lexicons_v22.json`)

8.3 Index Rebuild

```
corrector.rebuild_index(new_canonical_list)
```

Completely re-encodes all canonicals from scratch. This is idempotent — re-running always produces the same result. Used by pruning and extension tools to ensure consistency.

---

9. Evaluation Framework

The model is evaluated on four metrics, computed by `eval.py`:

9.1 Held-Out Variant Recall

- **What:** Unseen spelling variants → correct canonical?
- **Why:** The generalisation test. Can the model handle spellings it has never seen?
- **Metric:** Top-1 accuracy (with abstain), top-1 (raw nearest), top-3

9.2 Abstain Safety

- **What:** Real correct words that are NOT in the canonical set → % left unchanged
- **Why:** The anti-corruption test. The model must not "fix" words that are already correct.
- **Source:** Gold benchmark words from the 80-call evaluation set

9.3 Exact-Name Held-Out

- **What:** Held-out variants whose canonical is an entity name (person, place, org)
- **Why:** The decisive test. Names like Siddiqui/Chughtai are the highest-value corrections.
- **Metric:** Exact top-1 match accuracy

9.4 80-Call diff_words

- **What:** End-to-end accuracy on 80 production calls
- **How:** Compare v2 baseline (no model) vs v2 + contrastive model
- **Why:** The production metric. Does the model improve real-world output?

---

10. Integration with Production Pipeline

10.1 Activation

```
export PHONETIC=1                    # enable the model
export PHONETIC_THRESHOLD=0.90       # optional, default 0.90
```

10.2 Priority Chain in `hindi_to_roman_urdu.py`

The transliteration pipeline applies corrections in strict priority order:

```
1. Layer 1: Phoneme mapping (Devanagari → Roman)
2. Layer 2: Vowel-ending normalisation (regex)
3. Layer 3: Exact lexicon lookup (WORD_MAP)
   ├── Match found → return canonical (highest priority)
   └── No match →
       4a. Phonetic Contrastive Model (if PHONETIC=1)
           ├── Confident match → return canonical
           └── Abstain →
               4b. Rule-based Resolver (deprecated, if RESOLVER=1)
                   ├── Match → return canonical
                   └── No match → return word unchanged
```

The phonetic model NEVER overrides an exact lexicon hit. It only processes words the exact lexicon missed.

10.3 Integration Points

Consumer	How it uses the model
`hindi_to_roman_urdu.py`	Layer 4b fallback via `_PHONETIC.resolve_word()`
`retriever.py`	Acoustic biasing term normalisation
`benchmark_acoustic_biasing.py`	Benchmark runs with phonetic correction

---

11. Technical Specifications

11.1 Model Size

Component	Size/Count
Checkpoint file	~3.2 MB
Total parameters	~600K
Embedding layer	~30 × 96 = 2,880 params
GRU (2-layer, bidirectional)	~400K params
Projection head	512 × 128 + 128 = 65,664 params
Canonical index	(N, 128) float32

11.2 Runtime Performance

Metric	Value
Per-word latency (CPU)	< 1 ms
Model load time	< 0.5s
Memory footprint	~15 MB (model + index)
Device support	CPU, MPS (Apple Silicon), CUDA

11.3 Dependencies

```
torch          (PyTorch — inference only, no training deps needed)
```

No other dependencies are required at inference time. The `data.py` module's `Vocab` and `pad_batch` functions are the only internal imports.

---

12. Comparison with Rule-Based Resolver

Aspect	Rule-Based Resolver	Phonetic Contrastive Model
Held-out recall	~30%	~97%
Maintenance	O(N^2) — manually add rules	Add canonical, done
Generalisation	None — only programmed patterns	Generalises to unseen spellings
Abstain safety	Manual threshold tuning	Built-in 0.90 cosine threshold
Latency	~0.01 ms/word	~0.1 ms/word
Interpretability	Rules are readable	Embedding space — less readable
New canonical onboarding	Add variants manually	`add_canonical()` — no retrain

---

13. Limitations and Future Work

13.1 Current Limitations

1. **Accuracy-neutral on fuzzy metrics:** The model fixes spelling drift that fuzzy word-matching already forgives. Its value is primarily maintainability and consistency, not necessarily a score gain on diff_words.
2. **Character-level only:** Cannot handle multi-word phrase corrections (handled by the exact lexicon's phrase entries).
3. **Threshold sensitivity:** The 0.90 threshold is a single global number. Some word categories might benefit from category-specific thresholds.

13.2 Future Directions

- **Threshold per-category:** Different abstain thresholds for names vs common words
- **Online learning:** Continuously update the index as new corrections are confirmed
- **Multi-word support:** Extend to phrase-level embeddings for compound terms
- **Quantisation:** INT8 quantisation of the model for deployment on edge devices